

Mensch-Maschine-Schnittstelle: Analoge Spielzüge digitalisieren und verarbeiten am Beispiel von Schach

Besondere Lernleistung

Youcef Rezgui

Betreuende Lehrkraft: Sebastian Wilhelm

Gustav Heinemann Schule

Jahrgangsstufe 13

21. Juli 2026



Versicherung und Erklärung

Hiermit versichere ich, _____,

dass ich die besondere Lernleistung selbstständig ohne fremde Hilfe und ohne weitere Hilfsmittel
als den erlaubten und angegebenen erbracht habe.

Darüber hinaus erkläre ich, dass die vorliegende Arbeit und erbrachte Leistung nicht bereits
anderweitig in der Schule eingebracht wurde und verwendet worden ist.

Ort, Datum

Unterschrift

Inhaltsverzeichnis

1	Einleitung	4
1.1	Motivation und Relevanz des Themas	4
1.2	Zielsetzung der Arbeit	4
1.3	Aufbau und Vorgehensweise	5
2	Theoretische Grundlagen	5
2.1	Überblick Mensch-Maschine-Schnittstellen	5
2.2	Grundlagen Schach	5
2.3	Elektronische Komponenten	11
3	Entwicklung der Schnittstelle	13
3.1	Grundideen und Konzepte	13
3.2	Softwarekonzeption des GUI	14
3.3	Konzeption der Sensordatenerfassung	17
3.4	Speicherung der Züge und Organisation der Historie	17
4	Umsetzung und Implementierung	17
4.1	Hardwareentwicklung und Integration	17
4.2	Mikrocontroller Logik	19
4.3	Implementierung des GUI	22
5	Evaluation und Test	28
5.1	Test der Hardwarekomponenten	28
5.2	Test der Softwarekomponenten	29
5.3	Bewertung und Zusammenfassung der Testergebnisse	29
6	Zusammenfassung und Ausblick	30
7	Quellenverzeichnis	32

1 Einleitung

1.1 Motivation und Relevanz des Themas

Die Verbindung zwischen der physischen und der digitalen Welt spielt eine immer wichtigere Rolle in der modernen Technologieentwicklung. Besonders im Bereich der Mensch-Maschine-Interaktion wird angestrebt, analoge Prozesse effizient zu digitalisieren und in einer computergestützten Umgebung weiterzuverarbeiten. Ein besonders anschauliches Beispiel für ein solches System ist die Digitalisierung analoger Spielzüge im Schach.

Schach ist eines der ältesten und zugleich komplexesten Strategiespiele der Menschheitsgeschichte. Während es zahlreiche digitale Schachplattformen gibt, bleibt das klassische Schachbrett mit physischen Figuren für viele Spieler unverzichtbar. Allerdings besteht eine große Herausforderung darin, analoge Schachzüge automatisch und fehlerfrei zu erfassen, um diese für Analysen, Trainingszwecke oder als Schnittstelle für Online-Schachpartien zu nutzen.

Hier setzt diese Arbeit an: Durch die Integration von Sensoren in ein physisches Schachbrett sollen Spielzüge erkannt, verarbeitet und digitalisiert werden. Ein solches System könnte nicht nur das Spielerlebnis verbessern, sondern auch wertvolle Funktionen wie Partieverläufe speichern, Fehleranalysen durchführen oder sogar als Trainingshilfe für Schachspieler aller Leistungsstufen dienen.

Technisch gesehen ist die Umsetzung einer solchen Mensch-Maschine-Schnittstelle eine interdisziplinäre Herausforderung, da sowohl hardware- als auch softwareseitige Lösungen entwickelt werden müssen. Der Einsatz von Sensorik, Mikrocontrollern und Algorithmen zur Mustererkennung ist daher ein zentraler Bestandteil dieser Arbeit. Zu beachten ist, dass es sich bei dieser Arbeit um einen „**Proof of Concept**“, also einen exemplarischen Prototyp handelt und ich bei der Entwicklung daher den Schwerpunkt auf die Funktionalität der Schnittstelle gesetzt habe.

1.2 Zielsetzung der Arbeit

Das primäre Ziel dieser Arbeit ist die Entwicklung und Implementierung einer Mensch-Maschine-Schnittstelle, die in der Lage ist, analoge Schachzüge automatisch zu erfassen und digital weiterzuverarbeiten. Dazu soll ein Schachbrett mit geeigneten Sensoren ausgestattet werden, um die Positionen der Schachfiguren zu erfassen und ihre Bewegungen zu registrieren.

Um dieses Ziel zu erreichen, werden folgende Teilziele verfolgt:

- Entwicklung eines funktionalen Konzepts zur Erfassung der Schachzüge unter Berücksichtigung der technischen Möglichkeiten.
- Auswahl geeigneter Sensorik, die eine zuverlässige Identifikation der Figurenpositionen ermöglicht.
- Aufbau und Implementierung einer Hardwarelösung, die die Spielzüge in Echtzeit erkennt.
- Entwicklung einer Software, die die erfassten Daten interpretiert und eine sinnvolle Ausgabe generiert (z. B. in Form einer digitalen Partienotation oder Schnittstelle zu Schachsoftware).
- Test und Evaluierung des entwickelten Systems hinsichtlich Genauigkeit, Zuverlässigkeit und Praxistauglichkeit.

1.3 Aufbau und Vorgehensweise

Die Arbeit ist in mehrere Kapitel unterteilt, die die verschiedenen Schritte der Entwicklung systematisch behandeln werden. In folgender Arbeit werde ich daher diese einzelnen Teil Aspekte der Arbeit in Folgende Unterteilen:

1. **Theoretische Grundlagen**
2. **Entwicklung der Schnittstelle**
3. **Umsetzung und Implementierung**
4. **Evaluation und Test**
5. **Zusammenfassung und Ausblick**

2 Theoretische Grundlagen

In diesem Kapitel werden alle technischen, sowie theoretischen Grundlagen vermittelt, die im Vorfeld benötigt werden, um die spätere Methodik der Entwicklung des Projektes erfassen zu können.

2.1 Überblick Mensch-Maschine-Schnittstellen

Die Benutzerschnittstelle, auch als „Mensch-Maschine-Schnittstelle“ (MMS) bekannt, wird im Englischen als „Human Machine Interface“ (HMI) oder „Man Machine Interface“ (MMI) bezeichnet.¹ Sie ermöglicht es dem Nutzer nicht nur, eine Maschine zu steuern, sondern in vielen Fällen auch deren Status zu überwachen und aktiv in den Prozess einzugreifen. Die Rückmeldung („Feedback“) kann dabei auf verschiedene Weise erfolgen: Traditionell geschieht dies über Bedienpulte mit Signallampen, Anzeigefeldern und Tastern. Moderne Systeme setzen hingegen verstärkt auf softwaregestützte Visualisierung, die beispielsweise auf einem Bildschirm oder einem speziellen Terminal dargestellt wird.²

Ein Beispiel für eine Mensch-Maschine-Schnittstelle ist eine Mikrowelle. Der Nutzer gibt über Tasten oder ein Touchdisplay die gewünschte Zeit und Leistung ein, woraufhin die Mikrowelle das Essen erwärmt. Das „Feedback“ erfolgt akustisch durch einen Signalton am Ende des Vorgangs sowie visuell durch die Anzeige der verbleibenden Zeit auf dem Display.

2.2 Grundlagen Schach

Das Schachspiel zählt zu den ältesten Spielen der Welt und hat seinen vermuteten Ursprung im indischen Raum.³ Es wird auf einem quadratischen Schachbrett gespielt, das üblicherweise in abwechselnd helle und dunkle Felder unterteilt ist (zum Beispiel in hellbraun und dunkelbraun oder schwarz und weiß). Das Brett besteht aus insgesamt 64 Feldern, angeordnet in 8 Spalten, die von 1 bis 8 nummeriert sind, und 8 Reihen, die mit den Buchstaben a bis h bezeichnet werden.⁴

Im Schach gibt es sechs verschiedene Figuren: den Bauern, den Springer, den Läufer, den Turm, die Dame und den König. Jede Figur besitzt eigene Zugregeln, die festlegen, wie sie sich auf dem

¹wikipedia_Benutzerschnittstelle.

²elektrotechnik.

³wikipedia_Schachbrett.

⁴wikipedia_Schachbrett.

Brett bewegen – in der Schachsprache wird dies als „ziehen“ bezeichnet.

Das Spiel wird von zwei Spielern ausgetragen. Jeder Spieler erhält 16 Figuren, nämlich 8 Bauern, 2 Springer, 2 Läufer, 2 Türme, eine Dame und einen König. Die Figuren werden üblicherweise als schwarz und weiß bezeichnet, wobei die tatsächlichen Farben oft lediglich unterschiedliche Farbtöne (hell bzw. dunkel) sind, um eine Unterscheidung zu ermöglichen.⁵ Die Spieler ziehen abwechselnd. In jedem Zug kann eine eigene Figur auf dem Spielfeld bewegt werden. Es ist weder möglich, freiwillig auf einen Zug zu verzichten, noch einen bereits getätigten Zug rückgängig zu machen.⁶ Figuren der jeweils gegnerischen Farbe können „geschlagen“ werden, das heißt, sie werden aus dem Spiel entfernt. Eine Figur kann nur dann geschlagen werden, wenn sie von einer gegnerischen Figur bedroht wird, also auf einem Feld steht, auf das diese Figur ziehen kann.⁷

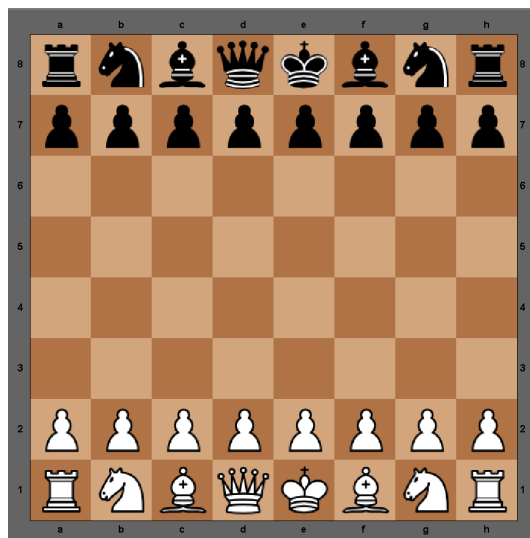


Abbildung 1: Grundstellung des Schachspiels. (eigens erstellte Abbildung)

Wie in Abbildung 1 dargestellt, ist die Grundaufstellung im Schach folgendermaßen: Die erste und die letzte Reihe werden als „Grund-Reihe“ bezeichnet.⁸ Zu Beginn des Spiels befinden sich die Türme auf den Feldern „a1“ und „h1“, die Springer auf den Feldern b1 und g1, die Läufer auf c1 und f1, die Dame auf d1 sowie der König auf e1 für die weiße Seite. Entsprechend ist die Aufstellung für Schwarz auf der achten Reihe identisch. Die acht Bauern beider Seiten werden jeweils auf der zweiten (für Weiß) und der siebten Reihe (für Schwarz) positioniert.

Im Schach bewegt sich jede der sechs Figuren auf eine eigene, festgelegte Weise. Dabei gelten folgende Regeln. Bauern ziehen grundsätzlich ein Feld geradeaus vorwärts. Beim ersten Zug eines Bauern kann er auch zwei Felder vorgehen, sofern beide frei sind (siehe Abbildung 2).

⁵Schachbegriffe.

⁶Schachregeln.

⁷wikipedia_schachfiguren.

⁸wikipedia_Schachbrett.

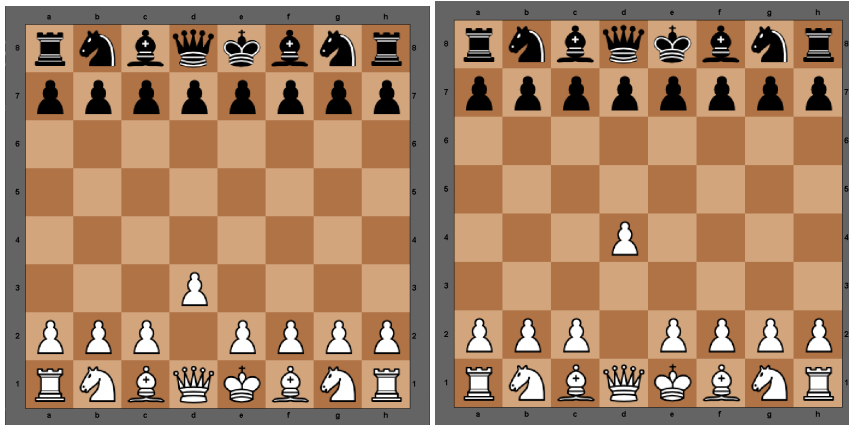


Abbildung 2: Der weiße Bauer zieht von d2 nach d3 und zieht beim erste Zug des Bauern von d2 nach d4. (eigens erstellte Abbildung)

Erreicht ein Bauer die gegnerische Grundreihe, erfolgt eine Umwandlung (Bauernumwandlung) in eine beliebige andere Figur (außer in den König).⁹

Der Springer bewegt sich in einem charakteristischen L-förmigen Muster, das heißt zwei Felder in eine Richtung und dann ein Feld rechts oder links davon. Er ist die einzige Figur, die über andere Figuren hinweg springen darf, unabhängig davon, ob diese eigene oder gegnerische Figuren sind.¹⁰

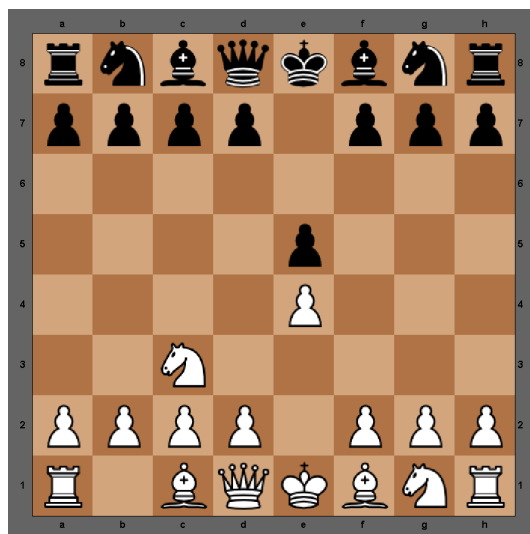


Abbildung 3: Der weiße Springer zieht von b1 nach c3. (eigens erstellte Abbildung)

Läufer ziehen diagonal über beliebig viele freie Felder und bleiben stets auf Feldern derselben Farbe, wie sie zu Beginn stehen (siehe Abbildung 3).¹¹,

⁹Schachregeln.

¹⁰Schachregeln.

¹¹Schachregeln.

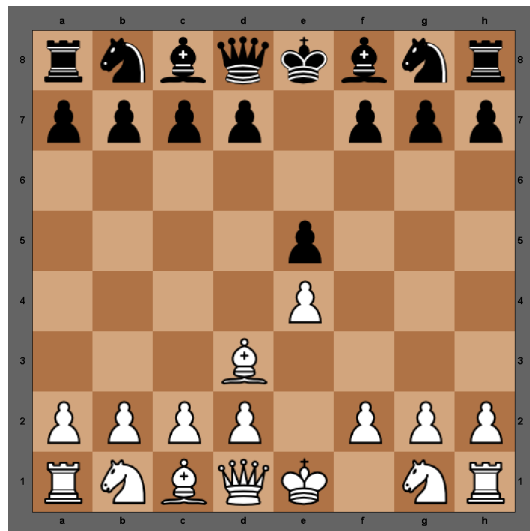


Abbildung 4: Der weiße Läufer zieht von f1 nach d3. (eigens erstellte Abbildung)

Türme bewegen sich horizontal oder vertikal über beliebig viele freie Felder (siehe Abbildung 5).¹²

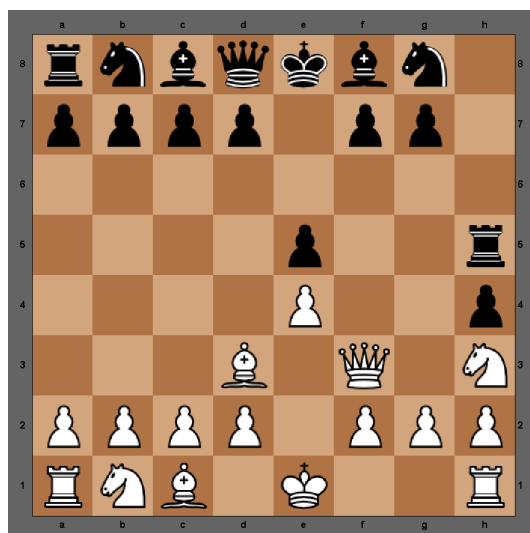


Abbildung 5: Der schwarze Turm zieht von h8 nach h5. (eigens erstellte Abbildung)

Die Dame kombiniert die Zugmöglichkeiten von Turm und Läufer. Sie kann sich sowohl horizontal, vertikal als auch diagonal über beliebig viele freie Felder bewegen (Siehe Abbildungen 6).¹³

¹²Schachregeln.

¹³Schachregeln.

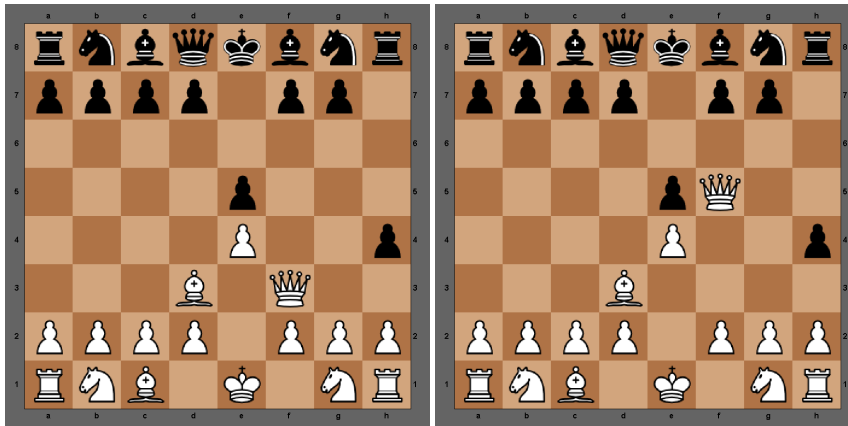


Abbildung 6: Die weiße Dame, zieht von d1 nach f3 und weiße Dame, zieht von f3 nach f5. (eigens erstellte Abbildung)

Der König ist die wichtigste Figur im Schachspiel und kann sich in jede Richtung bewegen – sei es horizontal, vertikal oder diagonal –, jedoch stets nur um ein Feld. Dabei ist er besonderen Regeln unterworfen: Er darf kein Feld betreten, das von einer gegnerischen Figur bedroht wird. Gerät der König selbst in Gefahr, also ins Schach, muss der Spieler ihn in diesem Zug aus der Bedrohung befreien.¹⁴

Eine besondere Bewegung des Königs ist die Rochade, ein Zug, bei dem er gemeinsam mit einem der Türme seine Position verändert. Damit dieser Zug erlaubt ist, müssen mehrere Bedingungen erfüllt sein: Weder der König noch der beteiligte Turm dürfen sich zuvor bewegt haben, zwischen ihnen darf keine andere Figur stehen, und der König darf sich weder im Schach befinden noch während der Rochade ein bedrohtes Feld überqueren oder darauf landen.¹⁵

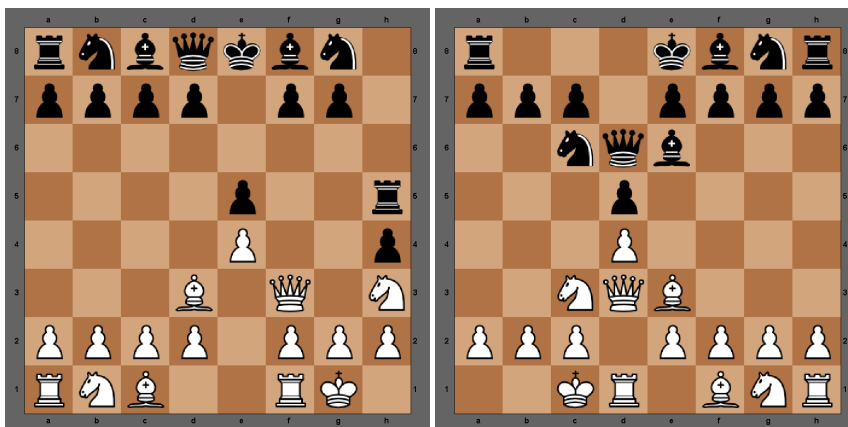


Abbildung 7: Lange und kurze Rochade (eigens erstellte Abbildung)

Mit Ausnahme des Springers kann keine Figur über andere Figuren hinwegziehen. Es ist auch nicht möglich auf ein Feld zu ziehen, welches von einer eigenen Figur besetzt wird. Es ist jedoch erlaubt, auf ein Feld zu ziehen, das von einer gegnerischen Figur belegt ist – in diesem Fall wird die gegnerische Figur geschlagen und aus dem Spiel entfernt. Eine Besonderheit stellt jedoch der Bauer dar. Zum Schlagen zieht der Bauer nämlich ein Feld diagonal vorwärts statt wie sonst ein Feld geradeaus vorwärts (siehe Abbildung 8).

¹⁴Schachbegriffe.

¹⁵Schachbegriffe.

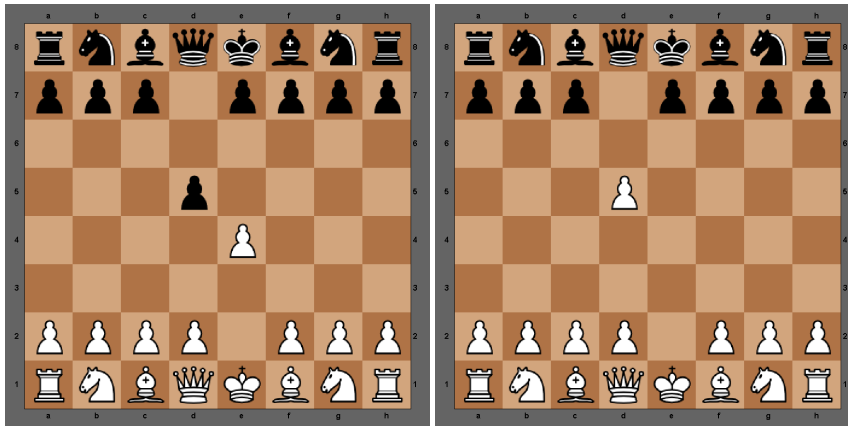


Abbildung 8: der weiße Bauer schlägt Schwarz auf Diagonal auf d5. (eigens erstellte Abbildung)

Zudem gibt es die **En Passant**-Regel, zu deutsch „**im Vorübergehen**“. Diese gilt, wenn ein Bauer einen Doppelschritt macht und neben einem gegnerischen Bauern steht, dann kann dieser Bauer hinter den gegnerischen Bauern Ziehen und dabei schlagen (siehe Abbildung 9).¹⁶

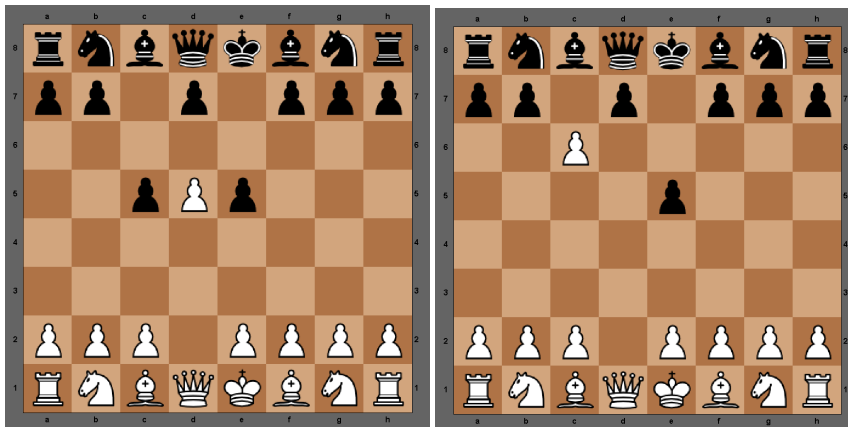


Abbildung 9: Beispielstellung für En Passant: Der weiße Bauer schlägt den schwarzen Bauern auf c5 im Vorübergehen. (eigens erstellte Abbildung)

Darüber hinaus besitzt die weiße Seite den „**Anzugsvorteil**“, was bedeutet, dass Weiß den ersten Zug ausführen darf und somit von Beginn an die Initiative übernimmt.¹⁷

Das Ziel des Spiels besteht darin, den gegnerischen König in ein sogenanntes „**Schachmatt**“¹⁸ zu setzen. Dies bedeutet, dass der König angegriffen wird (im „Schach“ steht) und keine Möglichkeit hat, der Bedrohung durch Ausweichen, Schlagen der angreifenden Figur oder Decken des Königs durch eine eigene Figur zu entkommen. Wer den gegnerischen König Schachmatt setzt, gewinnt die Partie.¹⁹ Eine Partie kann auch im Unentschieden oder auch „**remis**“ Enden, wenn z.B 50 Züge lange weder ein Bauer gespielt wurde noch eine Figur geschlagen wurde, oder aber ein Schachmatt nicht mehr möglich ist, Zwei König sind nur noch auf dem Feld übrig.

¹⁶Schachregeln.

¹⁷wikipedia_schachspiel.

¹⁸Schachregeln.

¹⁹wikipedia_schachspiel.

2.3 Elektronische Komponenten

Für die Sensorik kamen bei der Auswahl des Projekts viele verschiedene Bauteile in Betracht. Im Folgenden werden die zentralen Komponenten detailliert erläutert, ihre Funktionsweise beschrieben und durch Schaubilder visualisiert. Zu diesen Komponenten zählen unter anderem die Hallsensoren, der Multiplexer und der Arduino Mega.

Ein **Mikrocontroller** ist ein kompakter integrierter Schaltkreis, der alle Komponenten eines minimalen Computersystems vereint: Einen Prozessor (CPU), programmierbaren Speicher (Flash/ROM) sowie konfigurierbare Ein- und Ausgänge (I/O-Pins). Im Gegensatz zu Mikroprozessoren benötigen Mikrocontroller keine externen Komponenten für den Betrieb und sind speziell für Steuerungs- und Regelungsaufgaben optimiert. Sie werden in eingebetteten Systemen eingesetzt, wo sie Sensordaten erfassen, verarbeiten und darauf basierend z.B Motoren oder LEDs ansteuern.

Der **Arduino Mega rev 3** ist eine Mikrocontroller-Platine, die auf dem **ATmega2560**-Chip basiert. Als programmierbare Steuereinheit kombiniert er Rechenleistung, Speicher und anpassbare Ein-/Ausgänge in einem kompakten System. Mit einer Taktfrequenz von 16 MHz, **54 digitalen I/O-Pins** und **16 analogen Eingängen** eignet er sich besonders für Echtzeit-Anwendungen wie die Sensordatenverarbeitung in diesem Projekt. In Abbildung 10 ist eine schematische Darstellung des Arduino Mega rev 3 zu sehen.

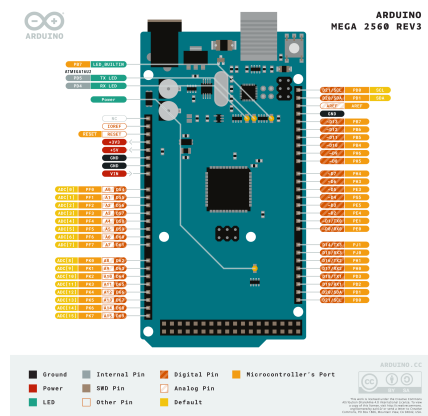


Abbildung 10: Übersicht der Arduino Mega-Platine mit den wichtigsten Anschlüssen **Arduino_Mega**.

Die im Projekt zur primären Erfassung eingesetzten **Hallsensoren** basieren auf dem **Hall-Effekt**. Wird ein stromdurchflossener Leiter in ein homogenes Magnetfeld gebracht, wirkt auf die beweglichen Elektronen die Lorentz-Kraft. Diese bewirkt, dass sich die Elektronen seitlich sammeln, so dass an den gegenüberliegenden Seiten des Leiters ein Ladungsungleichgewicht entsteht. Dieses Ungleichgewicht erzeugt ein internes elektrisches Feld, das sich so lange aufbaut, bis die Kraft des elektrischen Feldes die Lorentz-Kraft ausgleicht. Das Ergebnis ist eine messbare Spannung, die Hall-Spannung U_H , welche sich senkrecht zur Strom- und Magnetfeldrichtung befindet. Mathematisch lässt sich die Hall-Spannung folgendermaßen ausdrücken:

$$U_H = R_H \cdot \frac{I \cdot B}{d},$$

Diese Eigenschaft wird genutzt, um Magnetfelder präzise zu messen.²⁰ Im speziellen Fall der eingesetzten digitalen unipolaren Hallsensoren reagiert das Bauteil ausschließlich auf Magnetfelder einer bestimmten Polarität, beispielsweise nur auf den Nordpol. Sobald der Magnetfeldstärke-Schwellwert überschritten wird, ändert der Sensor seinen Ausgangszustand von HIGH zu LOW oder umgekehrt. Dadurch entsteht ein klar abgegrenztes digitales Signal, das direkt von einem Mikrocontroller ausgelesen werden kann. Ein schematischer Aufbau eines solchen Sensors ist in Abbildung 11 dargestellt.

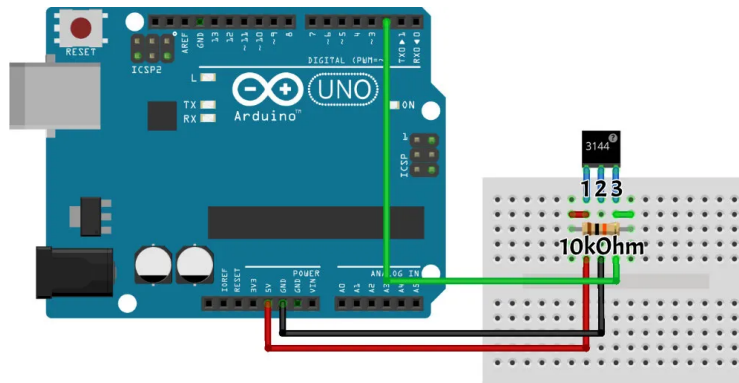


Abbildung 11: Schematische Darstellung eines digitalen unipolaren Hallsensors. Quelle: **Hallsensoren_Arduino_projekt**

In Anwendungen, in denen viele Sensoren verwendet werden, ist es oft unpraktisch, jeden Sensor direkt an den Mikrocontroller anzuschließen, da die Anzahl der verfügbaren Ein- und Ausgänge begrenzt ist. Ein **Multiplexer (MUX)** löst dieses Problem, indem er 16 Eingangssignale sequenziell über einen gemeinsamen Ausgang ausgibt. Dafür werden vier Adressleitungen (S_0 bis S_3) benötigt, da $2^4 = 16$ mögliche Kombinationen existieren. Ein 4-Bit-Steuercode legt fest, welcher der 16 Kanäle (C_0 bis C_{15}) aktiviert wird. Dies reduziert den Anschlussaufwand erheblich, da nur ein Ausgangssignal (SIG) und vier Steuerleitungen erforderlich sind.

0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	0
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	0
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	0

1

S3 S2 S1 S0

Abbildung 12: Adresstabelle des 16:1-Multiplexers. Die vier Adressbits (S_3 bis S_0) bestimmen den aktiven Kanal C_0 bis C_{15} , **Mux_tabelle**

Der **CD74HC4067** ist ein typischer 16:1-Multiplexer für Mikrocontroller-Anwendungen. Die Tabelle in Abbildung 12 zeigt die Zuordnung zwischen dem 4-Bit-Code und den Kanälen. Beispielsweise wird Kanal C_9 aktiviert, wenn $S_3 = 1$, $S_2 = 0$, $S_1 = 0$, und $S_0 = 1$ (Binärcode 1001). Der

²⁰Hallsensoren_physik.

SIG-Pin gibt dann das Signal des Sensors an C_9 aus. Der **EN**-Pin (*Enable*) steuert die Aktivierung des Multiplexers: Bei **GND**-Pegel ist das Modul aktiv, bei **VCC** deaktiviert.

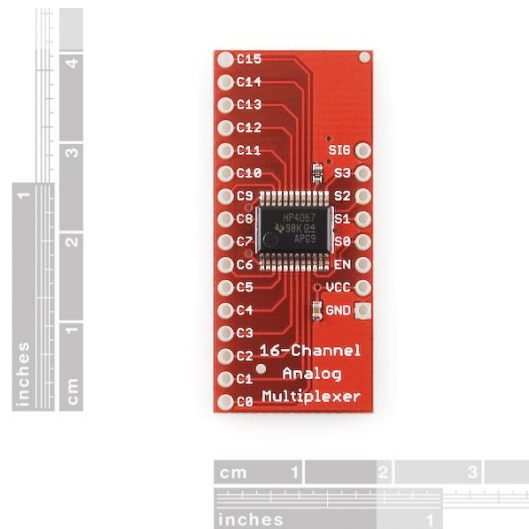


Abbildung 13: Schaltplan des 16:1-Multiplexers CD74HC406, **Analog_Digital_MUX**

3 Entwicklung der Schnittstelle

3.1 Grundideen und Konzepte

Die Entwicklung der Schnittstelle begann mit der Auswahl eines geeigneten physischen Schachbretts als Grundlage. Ich entschied mich für ein klassisches Holzschachbrett, da es nicht nur stabil, sondern auch ästhetisch ansprechend war. „Um einen besseren Eindruck vom verwendeten Schachbrett zu vermitteln, ist in Abbildung 14 eine Darstellung zu sehen. Damit es jedoch als digitale Schnittstelle dienen konnte, musste es mit Sensoren ausgestattet werden, die die Positionen der Figuren erfassen konnten.“

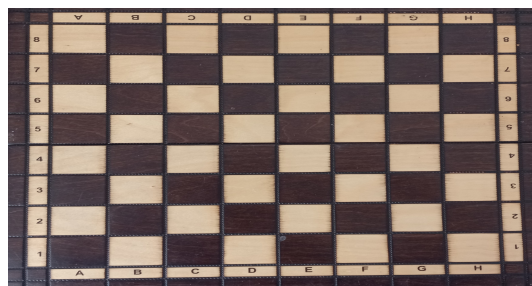


Abbildung 14: Ein herkömmliches Holzschachbrett, das als Basis für die Schnittstellenentwicklung gewählt wurde.

Mein Konzept sah vor, das Schachbrett durch eine eigens vorgenommene Verkabelung mit Sensoren auszustatten. Dafür platzierte ich unter jedem der 64 Felder gezielt eine Lücke in der Kupferkabelmatrix. An diesen definierten Stellen entfernte ich die Isolierung der Leitungen und lötete jeweils einen Sensor an.

Die Wahl der Hallsensoren war das Ergebnis einer Abwägung verschiedener Technologien. Ursprünglich hatte ich auch an RFID-Scanner gedacht, da sie die Möglichkeit geboten hätten, nicht nur

die Positionen, sondern auch die Identität der Figuren zu erkennen. Allerdings hätten RFID-Scanner ein erhebliches Platzproblem mit sich gebracht. Jeder Scanner benötigt eine gewisse Fläche, und die gegenseitige Beeinflussung der Scanner durch Interferenzen auf engstem Raum hätte es nahezu unmöglich gemacht, verlässliche Messungen durchzuführen²¹.

Hallsensoren hingegen boten eine einfachere und kompaktere Lösung. Sie messen magnetische Felder, was bedeutete, dass jede Figur lediglich mit einem kleinen Magneten in der Basis ausgestattet werden musste, um von den Sensoren erkannt zu werden. Diese Lösung war platzsparend und technisch leicht umsetzbar. Ein entscheidender Nachteil dieser Methode war jedoch, dass die Sensoren lediglich das Vorhandensein einer Figur auf einem bestimmten Feld erkennen konnten, nicht jedoch die Identität der Figur selbst. Während RFID-Scanner theoretisch in der Lage gewesen wären, zwischen König, Dame oder Bauer zu unterscheiden, war dies mit Hallsensoren nicht ohne Weiteres möglich. Dieses Problem sollte später noch eine Herausforderung darstellen, insbesondere bei der genauen Rekonstruktion von Spielzügen²².

Um die fehlende Unterscheidbarkeit der Figuren zumindest im Spiel zu kompensieren, suchte ich aus dem Internet nach einem 3D-Modell für Schachfiguren.²³ Ich passte dieses Modell so an, dass Magneten an den Figuren befestigt werden konnten. Diese angepassten Schachfiguren sollen dann als echte Spielfiguren verwendet werden.

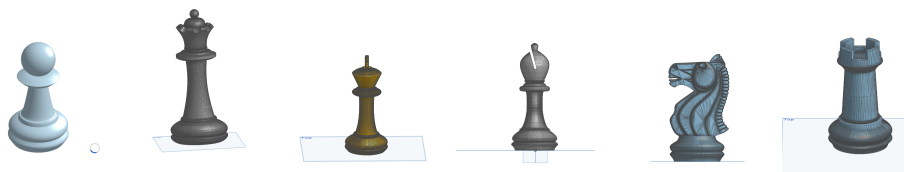


Abbildung 15: 3D-Modelle der Schachfiguren mit angepasster Befestigung für Magneten
3D_Figuren

3.2 Softwarekonzeption des GUI

Neben der Hardware musste auch eine Software entwickelt werden, die die erfassten Daten verarbeiten und visualisieren konnte. Da ich bereits Erfahrungen mit Java aus dem Unterricht hatte, entschied ich mich, die grafische Benutzeroberfläche (GUI) in dieser Sprache zu schreiben.

Die Kommunikation zwischen dem Schachbrett und der Software sollte über eine serielle Schnittstelle erfolgen. Hierfür verwendete ich die Bibliothek `jSerialComm`²⁴, die eine einfache Kommunikation mit dem Arduino ermöglichte. Der Arduino wiederum würde für das Auslesen der Hallsensoren verantwortlich sein und sendet die gesammelten Daten über die serielle Schnittstelle an die Java-Anwendung, wo sie weiterverarbeitet wurden. Zusätzlich nutzte ich die Bibliothek `jlayer-1.0.1`, um Soundeffekte in die Benutzeroberfläche zu integrieren.²⁵

Das Java-Projekt folgt einem modularen Aufbau und besteht aus mehreren zentralen Komponenten. Eine dieser Komponenten kümmert sich um die serielle Kommunikation, bei der die Bibliothek

²¹Rfid.

²²reddit_schachbrett.

²³3D_Figuren.

²⁴arduino_forum_java_serial.

²⁵youtube_Jlayer.

`JSerialComm` verwendet wird, um die Daten des Arduinos in Echtzeit zu empfangen. Ein weiteres Modul enthält die Schachlogik, die durch Klassen wie `ChessApp.java`, `Chessboard.java` und `ChessMove.java` den aktuellen Spielzustand verwaltet, Züge validiert und die Schachbrett-position speichert. Darüber hinaus wurden separate Klassen für die einzelnen Schachfiguren (z.B. `Pawn.java`, `Knight.java`, `Bishop.java` usw.) implementiert, die die spezifischen Zugregeln jeder Figur festlegen.

Für die grafische Darstellung des Schachspiels werden die Java-Bibliotheken **AWT (Abstract Window Toolkit)** und **Swing** verwendet. AWT bietet grundlegende GUI-Komponenten wie Fenster, Buttons und Event-Handling, ist jedoch plattformabhängig. Swing, eine Erweiterung von AWT, nutzt hingegen plattformunabhängige Komponenten und ermöglicht anpassbare Oberflächen mit erweiterten Features wie `JPanel`, `JFrame` oder `JButton`.

Die Klasse `GamePanel.java` erbt von `JPanel` und dient zur Visualisierung des Schachbretts. Sie überschreibt die `paintComponent`-Methode, um das Spielfeld mittels `Graphics2D` zu zeichnen (z. B. Felder, Figuren). Mithilfe von `MouseListener` wird die Benutzerinteraktion (Klicken, Ziehen von Figuren) implementiert. Swing-Komponenten wie `JMenuBar` oder `JDialog` können zusätzlich Menüs oder Dialogfenster bereitstellen.

Für die Audiowiedergabe (z. B. Soundeffekte bei Zugausführung) kommt die Bibliothek `jlayer-1.0.1` zum Einsatz, die MP3-Dateien dekodieren kann.²⁶ Die Integration erfolgt typischerweise über die Klasse `Player` aus dem Paket `javazoom.jl.player`.

Ein Tutorial zur Grundlagenimplementierung eines Schachspiels in Java, das AWT/Swing und Objektorientierung erklärt, ist hier referenziert.²⁷

Zur Veranschaulichung der geplanten Softwarearchitektur habe ich ein UML-Diagramm erstellt, das den geplanten Ablauf des Projekts modelliert (siehe Abbildung 16). Das Diagramm wurde über den Verlauf des Projekts hinweg angepasst und zeigt, wie die verschiedenen Module in Beziehung zu einander stehen.

²⁶<https://www.javazoom.net/javayer/javayer.html>

²⁷<https://example.com/chess-tutorial>

3.3 Konzeption der Sensordatenerfassung

Die Grundidee der Sensordatenerfassung ist es, die Hallsensor-Matrix, die über einen Multiplexer angeschlossen ist, systematisch auszulesen und die erfassten Daten an die übergeordnete Software zu übermitteln, also dem GUI. Der Code aktiviert dabei in regelmäßigen Abständen jeweils einen Kanal des Multiplexers, liest den entsprechenden Sensorwert aus und überträgt diesen über die serielle Schnittstelle

Konkret funktioniert dies so, dass der Arduino über vier Adressleitungen den gewünschten Kanal des Multiplexers auswählt. Sobald der jeweilige Hallsensor, der unter einem bestimmten Spielfeld platziert aktiviert wird, soll der Sensorwert ausgelesen und über einen Datenausgang ausgegeben werden. Dieser Wert zeigt an, ob ein Magnet (und damit eine Schachfigur) von dem Sensor erkannt wurde oder nicht. Anschließend wird dieser Wert zusammen mit der Kanalinformation (also der Position des Sensors auf dem Schachbrett) an die Java-Anwendung gesendet. Die Datenübertragung erfolgt über die serielle Schnittstelle, sodass die erfassten Informationen in Echtzeit verarbeitet und auf der grafischen Oberfläche angezeigt werden können.

3.4 Speicherung der Züge und Organisation der Historie

Zu Beginn meiner Überlegungen zur Speicherung der Züge im Spiel zog ich in Erwägung, eine Liste zu verwenden – eine Datenstruktur, die sich besonders gut dafür eignet, alte Stellungen jederzeit erneut anzeigen zu können und dabei nur geringen Overhead mitbringt. Bei weiteren Recherchen stieß ich jedoch auf ein Video²⁸, in dem effiziente Methoden zur Organisation der Zughistorie im Schach vorgestellt wurden. Dort wurde ein Ansatz präsentiert, bei dem Züge mittels Zobrist-Hashing gespeichert werden – eine Methode, die den Spielzustand in kompakter Form abbildet.

Der wesentliche Vorteil des Zobrist-Hashing-Ansatzes besteht darin, dass für jede mögliche Stellung auf dem Schachbrett ein eindeutiger Hash-Wert berechnet wird, der den aktuellen Zustand der Partie exakt widerspiegelt. Dies ermöglicht nicht nur eine schnelle und effiziente Speicherung, sondern auch einen zügigen Vergleich verschiedener Spielstände. Nach weiterer Recherche^{29,30} entschied ich mich, diesen Ansatz in meinem Projekt zu integrieren, um die gesamte Historie der Schachpartie effizient zu verwalten und frühere Stellungen rasch überprüfen zu können.

4 Umsetzung und Implementierung

4.1 Hardwareentwicklung und Integration

Der erste Schritt bestand darin, das physische Schachbrett auszumessen, um die Maße eines Spielfeldes zu ermitteln. So konnte ich sicherstellen, dass die Sensoren exakt unter den vorgesehenen Feldern positioniert wurden.

Unter jedem der 64 Felder platzierte ich Hallsensoren, die den Zustand „HIGH“ bzw. „LOW“ melden, also ob eine Figur auf dem Feld ist oder nicht. Die Verkabelung erfolgte über zwei sepa-

²⁸[zobrist_hashing_youtube](#).

²⁹[wikipedia_zobrist](#).

³⁰[Comptuerphile](#).

rate Breadboards, die jeweils die Stromversorgung für die rechte und linke Seite des Schachbretts übernahmen. Rote Kabel führten die Spannung (5 V), während schwarze Kabel als Ground dienten. Besonders aufwendig war das präzise Abisolieren und Anbringen der Kabel, da eine fehlerhafte Verkabelung zu unzuverlässigen Sensorwerten führen konnte.

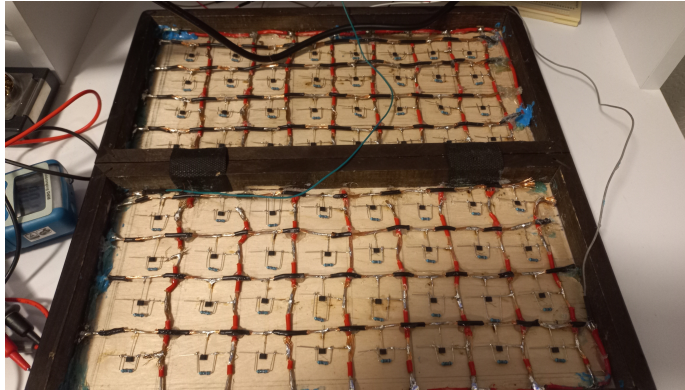


Abbildung 17: Vollendete Hallsensoren-Matrix

Wie zuvor geplant setzte ich beim auslesen der Sensor Daten MUX ein (siehe S.15), dabei musste ich sicherstellen, dass die 4-Bit-Steuercodes korrekt an die jeweiligen Pins des MUX übertragen wurden. Während des Aufbaus traten unerwartete Probleme auf: Zwei der eingesetzten MUX-Module waren defekt, was mich zwang, Ersatzkomponenten zu besorgen und die Schaltung anzupassen. Zudem wechselte ein MUX ständig unkontrolliert die Signale zwischen seinen Kanälen. Messungen ergaben, dass ein Kurzschluss zwischen den Leiterbahnen der Adressleitungen vorlag. Ein weiteres Problem war, dass eine der Adressleitungen „floating“ war, da die Lötstelle nicht richtig kontaktierte. Diese fehlerhafte Verbindung führte dazu, dass falsche Kanaladressen codiert wurden und somit falsche Sensorwerte übertragen wurden. Nach Behebung dieser Fehler verlötete ich die einzelnen Multiplexer Pins wieder richtig an und verkabelte dann die einzelnen Kanäle mit den Sensoren wie in Abbildung 18 zu sehen ist:

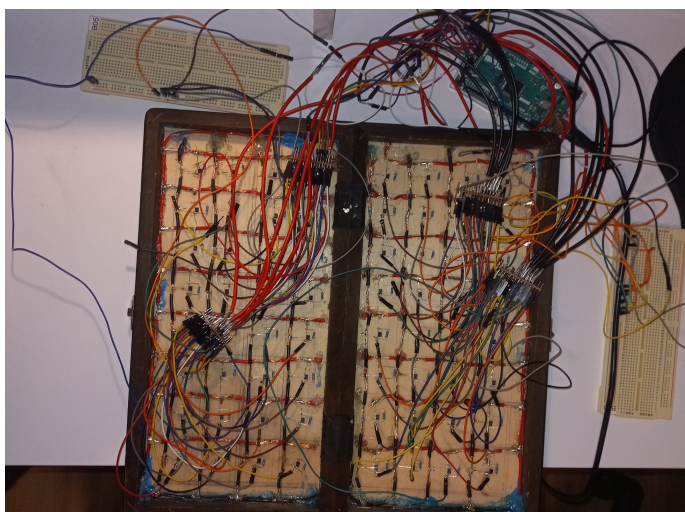


Abbildung 18: Vollendeter Prototyp der Schnittstelle

4.2 Mikrocontroller Logik

Die Software Logik des Mikrocontrollers ist relativ simpel. Zunächst wird ein zweidimensionales Array `muxPins` definiert, das für jeden der vier Multiplexer die zugehörigen Pin-Nummern enthält. Dabei werden die ersten vier Elemente eines jeden Arrays als Ausgänge zur Adressierung genutzt und das fünfte Element dient als Eingang für den Sensorsignalwert. Gleichzeitig wird ein Array `previousSensorValues` initialisiert, in dem die zuletzt gelesenen Sensorwerte gespeichert werden.

```
1  int muxPins[4][5] = {
2      {50, 51, 52, 53, 35}, // MUX1: Reihen 1-4, Spalten E-H
3      {46, 47, 48, 49, 34}, // MUX2: Reihen 5-8, Spalten E-H
4      {42, 43, 44, 45, 33}, // MUX3: Reihen 5-8, Spalten A-D
5      {38, 39, 40, 41, 32} // MUX4: Reihen 1-4, Spalten A-D
6  };
7
8  int previousSensorValues[64] = {0};
```

Listing 1: Definition der Multiplexer-Pins und Initialisierung des Sensorwertspeichers

Im nächsten Schritt werden mehrere Hilfsfunktionen definiert. Die Funktion `setMuxChannel` wählt den gewünschten Kanal des Multiplexers aus, indem sie die digitalen Ausgänge entsprechend des Binärwerts des Kanal-Parameters setzt. Ein kurzes Delay stellt sicher, dass sich die Signale stabilisieren.

```
1  void setMuxChannel(const int sPins[4], int channel) {
2      for (int i = 0; i < 4; i++) {
3          digitalWrite(sPins[i], bitRead(channel, 3 - i)); // S3=LSB
4      }
5      delayMicroseconds(5);
6  }
```

Listing 2: Funktion `setMuxChannel` zur Kanalauswahl

Die Funktion `readSensor` liest anschließend den Sensorsignalwert vom angegebenen Eingangspin aus. Dabei wird HIGH als 0 und LOW als 1 interpretiert, was in dieser Logik der Besetzungsstatus des Feldes ist.

```
1  int readSensor(int sigPin) {
2      return digitalRead(sigPin) == HIGH ? 0 : 1;
3  }
```

Listing 3: Funktion `readSensor` zum Auslesen des Sensorsignals

Um den aktuellen Zustand des Schachbretts über den Serial Monitor auszugeben, wird die Funktion `printBoard` verwendet. Diese gibt die 8x8-Anordnung als ASCII-Art aus, wobei besetzte Felder durch ein „X“ und leere Felder durch einen Punkt (".") dargestellt werden. Sie dient hauptsächlich zum debuggen von möglicherweise auftretenden Problemen

```

1  void printBoard(int sensorValues[64]) {
2      Serial.println("\nSchachbrett-Zustand:");
3      Serial.println("  A B C D E F G H");
4      Serial.println(" +-----+");
5      for (int row = 0; row < 8; row++) {
6          Serial.print(8 - row); // Zeilennummer
7          Serial.print("|");
8          for (int col = 0; col < 8; col++) {
9              int index = row * 8 + col;
10             Serial.print(sensorValues[index] ? "X " : ". "); // X =
11                 Figur, . = Leer
12         }
13         Serial.println();
14     }
15     Serial.println();
16 }

```

Listing 4: Funktion printBoard zur Darstellung des Schachbrettzustands

Im setup werden der serielle Anschluss initialisiert und alle benötigten Pins der Multiplexer als Ausgänge (für die Kanalauswahl) bzw. als Eingang (für das Sensorsignal) konfiguriert. Der Arduino gibt anschließend eine Bestätigung über den Serial Monitor aus.

```

1  void setup() {
2      Serial.begin(115200);
3      Serial.println("Starte Setup...");
4
5      for (int mux = 0; mux < 4; mux++) {
6          for (int i = 0; i < 4; i++) {
7              pinMode(muxPins[mux][i], OUTPUT);
8          }
9          pinMode(muxPins[mux][4], INPUT);
10     }
11
12     Serial.println("Setup abgeschlossen!");
13 }

```

Listing 5: Setup-Funktion im Arduino-Code

In der loop-Funktion erfolgt die kontinuierliche Überwachung des Schachbretts. Zunächst werden alle 64 Sensorwerte ausgelesen, indem für jeden Multiplexer alle 16 Kanäle sequenziell aktiviert und der Sensorsignalwert abgefragt wird. Die Position des Sensors wird dabei anhand berechneter Zeilen- und Spaltenindizes ermittelt und im Array sensorValues abgelegt.

Nach dem Auslesen erfolgt ein Vergleich der aktuellen Sensorwerte mit den vorherigen Werten. Wird genau an zwei Stellen ein Unterschied festgestellt – ein Feld, das von „besetzt“ zu „leer“ wechselt, und ein Feld, das von „leer“ zu „besetzt“ wechselt –, so wird dies als gültiger Zug interpretiert. Der erkannte Zug wird dann in einem vorgegebenen Format (eingeleitet mit START, und abgeschlossen

mit (, END) über den Serial-Port ausgegeben. Abschließend werden die aktuellen Sensorwerte in das Array previousSensorValues kopiert, sodass der nächste Durchlauf den neuen Zustand als Referenz verwendet wird. Ein Delay von 500 Millisekunden den einzelnen Zyklen gewährleistet eine stabile Abtastrate.

```
1 void loop() {
2     int sensorValues[64] = {0};
3
4     // Sensordaten auslesen
5     for (int mux = 0; mux < 4; mux++) {
6         for (int channel = 0; channel < 16; channel++) {
7             setMuxChannel(muxPins[mux], channel);
8             int val = readSensor(muxPins[mux][4]);
9
10            // Positionsberechnung
11            int reihe = (mux < 2) ? (channel / 4) : 4 + (channel /
12                4);
13            int spalte = (mux % 2 == 0) ? (7 - (channel % 4)) : (3 -
14                (channel % 4));
15            int index = reihe * 8 + spalte;
16
17            sensorValues[index] = val;
18        }
19    }
20
21    // Schachbrett mit aktuellen Sensorwerten ausgeben zum Debuggen
22    printBoard(sensorValues);
23
24    // Bewegungserkennung: Vergleiche aktuelle Sensorwerte mit den
25    // vorherigen
26    int fromIndex = -1, toIndex = -1, diffCount = 0;
27    for (int i = 0; i < 64; i++) {
28        if (sensorValues[i] != previousSensorValues[i]) {
29            diffCount++;
30            if (previousSensorValues[i] == 1) {
31                fromIndex = i;
32            } else {
33                toIndex = i;
34            }
35        }
36    }
37
38    // Sende nur Zugdaten im erwarteten Format, wenn genau zwei
39    // Unterschiede festgestellt wurden
40    if (diffCount == 2 && fromIndex != -1 && toIndex != -1) {
41        Serial.print("Erkannter Zug: ");
42        Serial.print((fromIndex / 8) + 1);
43        Serial.print((char)('A' + (fromIndex % 8)));
44        Serial.print(" -> ");
```

```

41     Serial.print((toIndex / 8) + 1);
42     Serial.print((char)('A' + (toIndex % 8)));
43     Serial.println();
44
45     Serial.print("START,");
46     Serial.print((fromIndex / 8) + 1);
47     Serial.print(",");
48     Serial.print((char)('A' + (fromIndex % 8)));
49     Serial.print(",");
50     Serial.print((toIndex / 8) + 1);
51     Serial.print(",");
52     Serial.print((char)('A' + (toIndex % 8)));
53     Serial.println(",END");
54 }
55
56 // Sensorwerte aktualisieren
57 memcpy(previousSensorValues, sensorValues,
58         sizeof(previousSensorValues));
59 delay(500);

```

Listing 6: Die loop-Funktion zur Erfassung und Übertragung der Sensorwerte

4.3 Implementierung des GUI

In der Chessboard-Klasse wird die gesamte Schachlogik umgesetzt – angefangen bei der Anfangsaufstellung der Figuren bis hin zur Validierung von Sonderzügen wie Rochade, En Passant und Bauernumwandlung. In der Methode `movePiece` wird ein Zug ausgeführt, wobei zunächst geprüft wird, ob es sich um einen Spezialfall wie En Passant handelt. Anschließend wird der Zug in der internen Brettdarstellung durchgeführt, wobei der Zobrist-Hash zur schnellen Erkennung von Stellungshistorien aktualisiert wird. Gleichzeitig wird der Zug in der `moveHistory` gespeichert, sodass er später beispielsweise für die `undoLastMove`-Funktion verwendet werden kann.

```

1     public void movePiece(int fromRow, int fromCol, int toRow, int
2         toCol, boolean switchTurn) {
3         Piece movingPiece = board[fromRow][fromCol];
4         Piece captured = board[toRow][toCol];
5
6         // En Passant: Bei diagonalem Zug eines Bauern in ein leeres Feld
7         if (movingPiece instanceof Pawn && Math.abs(toCol - fromCol) ==
8             1 && board[toRow][toCol] == null) {
9             int capturedPawnRow = fromRow;
10            captured = board[capturedPawnRow][toCol];
11            updateZobrist(captured, capturedPawnRow, toCol);
12            board[capturedPawnRow][toCol] = null;
13        }
14
15        // Entferne die Figur von der alten Position aus dem Hash
16        updateZobrist(movingPiece, fromRow, fromCol);
17        if (captured != null) {
18            updateZobrist(captured, toRow, toCol);
19        }
20    }

```

```

17     }
18     // Speichere den Zug in der Historie
19     moveHistory.push(new ChessMove(movingPiece, fromRow, fromCol,
20         toRow, toCol, captured));
21     // Fuehre den Zug aus
22     board[toRow][toCol] = movingPiece;
23     board[fromRow][fromCol] = null;
24
25     // Aktualisiere den Hash der neuen Stellung
26     updateZobrist(movingPiece, toRow, toCol);
27
28     // Sonderlogik: Bauernumwandlung und Rochade
29     if (movingPiece instanceof Pawn) {
30         handlePawnPromotion(movingPiece, toRow, toCol);
31     }
32     if (movingPiece instanceof King && Math.abs(toCol - fromCol) ==
33         2) {
34         handleCastling(fromRow, fromCol, toRow, toCol);
35     }
36
37     // Spielerwechsel und weitere Statusupdates
38     if (switchTurn) {
39         switchPlayer();
40     }
41 }

```

Listing 7: Auszug aus movePiece in der Chessboard-Klasse

Das GamePanel steuert die Anzeige und Animation der Züge. In der Methode processMove wird zunächst sichergestellt, dass Züge momentan verarbeitet werden dürfen. Anschließend wird geprüft, ob sich an der Startposition tatsächlich eine Figur befindet. Ist dies der Fall, wird die Gültigkeit des Zugs überprüft, bevor bei einem gültigen Zug eine Animation gestartet und bei einem ungültigen Zug der Zielbereich rot markiert sowie eine Rückanimation ausgeführt wird.

```

1     public void processMove(int fromRow, int fromCol, int toRow, int
2         toCol) {
3         // Blockierung, falls Züge momentan nicht erlaubt sind
4         if (movesBlocked || chessboard.isPromotionPending()) {
5             System.out.println("Zug wird momentan nicht verarbeitet.");
6             return;
7         }
8
9         Piece movingPiece = chessboard.getBoard()[fromRow][fromCol];
10        if (movingPiece == null) {
11            System.out.println("Keine Figur an der Startposition!");
12            resetInvalidState();
13            repaint();
14            return;
15        }
16
17        // Prüfung, ob der Zug gültig ist (eigene Figur, regelkonforme
18        // Bewegung, etc.)

```

```

17         if (!movingPiece.isValidMove(toCol, toRow,
18             chessboard.getBoard())) {
19             markInvalidMove(fromRow, fromCol, toRow, toCol);
20             animateMove(movingPiece, fromRow, fromCol, toRow, toCol,
21                 true);
22             return;
23         }
24
25         // Animation und Soundausgabe bei gültigem Zug
26         animateMove(movingPiece, fromRow, fromCol, toRow, toCol, false);
27     }

```

Listing 8: Auszug aus der processMove-Methode in GamePanel

Die Kommunikation mit dem Arduino erfolgt über die `jSerialComm`-Bibliothek. Ein Datenlistener wird eingerichtet, der die eintreffenden Sensordaten in einen Puffer schreibt und erst verarbeitet, wenn ein vollständiger Nachrichtenblock (zwischen `START` und `END`) empfangen wurde. Sobald ein solcher Block vorliegt, wird dieser an die Methode `processCompleteSensorData` weitergeleitet und der Puffer zurückgesetzt.

```

1     serialPort.addDataListener(new SerialPortDataListener() {
2         @Override
3         public int getListeningEvents() {
4             return SerialPort.LISTENING_EVENT_DATA_AVAILABLE;
5         }
6
7         @Override
8         public void serialEvent(SerialPortEvent event) {
9             if (event.getEventType() ==
10                 SerialPort.LISTENING_EVENT_DATA_AVAILABLE) {
11                 try {
12                     String line = input.readLine();
13                     if (line == null) {
14                         return;
15                     }
16
17                     dataBuffer.append(line);
18
19                     if (dataBuffer.indexOf("START,") >= 0 &&
20                         dataBuffer.indexOf(",END") >= 0) {
21                         int startIndex = dataBuffer.indexOf("START,");
22                         int endIndex = dataBuffer.indexOf(",END") + 4;
23                         String completeMessage =
24                             dataBuffer.substring(startIndex, endIndex);
25                         processCompleteSensorData(completeMessage);
26
27                         dataBuffer.delete(0, endIndex);
28                     }
29                 } catch (Exception e) {
30                     e.printStackTrace();
31                 }
32             }
33         }
34     });

```

```

30     }
31 }));

```

Listing 9: Auszug aus der serialEvent-Methode in ArduinoConnector

Über die Methode `processSensorData` in der `GamePanel`-Klasse werden die vom Arduino empfangenen Daten interpretiert. Diese Methode erwartet einen String im Format „START, <v0>,<v1> ...,<v63>, END“ und validiert zunächst das Format. Nach dem Entfernen der Start- und Endmarkierungen wird der String anhand von Kommas in einzelne Werte zerlegt. Es werden genau vier Werte erwartet, die dann in interne Koordinaten umgerechnet werden, bevor der Schachzug über die Methode `processMove` verarbeitet wird.

```

1  /**
2   * Verarbeitet die vom Arduino empfangenen Daten.
3   * Erwartet wird ein String im Format:
4   * "START,<v0>,<v1>,...,<v63>,END"
5   *
6   * Bei genau vier Werten wird ein Zug angenommen.
7   *
8   * @param data Die empfangenen Daten als String.
9   */
10 public void processSensorData(String data) {
11     data = data.trim();
12
13     // Validiere das Datenformat
14     if (!data.startsWith("START,") || !data.endsWith(",END")) {
15         System.out.println("Ungültiges Datenformat: " + data);
16         return;
17     }
18
19     // Entferne "START," und ",END"
20     String inner = data.substring(6, data.length() - 4);
21     String[] parts = inner.split(",");
22
23     // Erwarte 4 Werte: FromRow, FromCol, ToRow, ToCol
24     if (parts.length != 4) {
25         System.out.println("Erwartet 4 Werte
26             (FromRow,FromCol,ToRow,ToCol), erhalten: " +
27             parts.length);
28         return;
29     }
30
31     try {
32         // Konvertiere die Werte in Integer
33         int fromRow = Integer.parseInt(parts[0].trim());
34         int fromCol = parts[1].trim().charAt(0) - 'A'; // Spalte
35             A=0, B=1, etc.
36         int toRow = Integer.parseInt(parts[2].trim());
37         int toCol = parts[3].trim().charAt(0) - 'A';
38
39         // Konvertiere visuelle Reihen (1-8) in interne Reihen (0-7)
40         int internalFromRow = 8 - fromRow;

```



```

38         int internalToRow = 8 - toRow;
39
40         // Verarbeite den Zug
41         processMove(internalFromRow, fromCol, internalToRow, toCol);
42
43     } catch (NumberFormatException | StringIndexOutOfBoundsException
44             e) {
45         System.err.println("Fehler beim Parsen der Zugdaten: " +
46                             e.getMessage());
47     }
48 }

```

Listing 10: Codeausschnitt: Verarbeitung der Sensordaten

Die ChessMove-Klasse dient zur Dokumentation einzelner Züge. Jedes Objekt dieser Klasse enthält alle wichtigen Informationen zum Zug, wie Start- und Zielposition sowie eine eventuell geschlagene Figur. Mit dieser Klasse wird jeder Zug in der Zughistorie gespeichert, was unter anderem für Funktionen wie das Rückgängigmachen von Zügen genutzt wird. Dadurch kann die Schnittstelle eine genaue Historie der Partie erstellen.

```

1  public class ChessMove {
2      private Piece piece;
3      private int fromRow, fromCol;
4      private int toRow, toCol;
5      private Piece capturedPiece;
6
7      public ChessMove(Piece piece, int fromRow, int fromCol, int
8                      toRow, int toCol, Piece captured) {
9          this.piece = piece;
10         this.fromRow = fromRow;
11         this.fromCol = fromCol;
12         this.toRow = toRow;
13         this.toCol = toCol;
14         this.capturedPiece = captured;
15     }
16
17     // Getter-Methoden...
18 }

```

Listing 11: Auszug aus der ChessMove-Klasse

Der Zobrist-Hash wird in der Chessboard-Klasse verwendet, um die Stellung des Schachbretts effizient zu repräsentieren und zu vergleichen. Hierbei wird für jede mögliche Kombination aus Farbe, Figurentyp und Position ein zufälliger Wert generiert. Der Gesamt-Hash der Stellung wird dann durch das bitweise XOR (^) der entsprechenden Werte aller auf dem Brett befindlichen Figuren berechnet. Bei einem Zug wird der Hash durch erneutes XORen der Werte der betroffenen Positionen aktualisiert. Dadurch können Änderungen schnell und effizient vorgenommen werden, und alte Stellungen werden in einer Historie gespeichert, um beispielsweise Wiederholungen zu erkennen.

```

1  private void initializeZobrist() {
2      Random rand = new Random();
3      zobristTable = new long[2][6][MAX_ROW * MAX_COL];

```

```

4      for (int color = 0; color < 2; color++) {
5          for (int type = 0; type < 6; type++) {
6              for (int pos = 0; pos < MAX_ROW * MAX_COL; pos++) {
7                  zobristTable[color][type][pos] = rand.nextLong();
8              }
9          }
10     }
11 }

```

Listing 12: Auszug aus der Zobrist-Initialisierung in Chessboard

Neben der Initialisierung existieren zwei wesentliche Methoden, die den Zobrist-Hash während des Spiels aktualisieren. Die Methode `updateZobrist` aktualisiert den aktuellen Zobrist-Hash, indem sie den Einfluss einer bestimmten Figur an einer bestimmten Position toggelt. Dabei wird zunächst überprüft, ob das übergebene Figurenobjekt `null` ist. Falls nicht, wird der Typ der Figur mittels `getPieceType(piece)` ermittelt und die Position linear als `pos = row * MAX_COL + col` berechnet. Anschließend wird der aktuelle Hash-Wert durch eine bitweise XOR-Operation mit dem Wert aus der vorinitialisierten `zobristTable` an der entsprechenden Stelle aktualisiert. Durch dieses Verfahren wird der Einfluss der Figur an der betreffenden Position entweder hinzugefügt oder entfernt, wobei bei doppelter Anwendung der ursprüngliche Zustand wiederhergestellt wird.

```

1      private void updateZobrist(Piece piece, int row, int col) {
2          if (piece == null) return;
3          int type = getPieceType(piece);
4          int pos = row * MAX_COL + col;
5          currentZobristHash ^= zobristTable[piece.getColor()][type][pos];
6      }

```

Listing 13: Methode `updateZobrist`

Die Methode `undoLastMove` macht den letzten Zug rückgängig und stellt sowohl das Brett als auch den Zobrist-Hash in den Zustand vor dem Zug zurück. Zunächst wird geprüft, ob die Zug-historie nicht leer ist. Falls ein Zug vorhanden ist, wird dieser mittels `pop()` entfernt und in einer Variablen gespeichert. Anschließend wird der Hash am Zielort des Zuges aktualisiert, um den Beitrag der Figur an dieser Position zu entfernen. Danach erfolgt die Rücksetzung des Zugs auf dem Brett: Die Figur wird an ihre ursprüngliche Position zurückgesetzt und eine eventuell geschlagene Figur wiederhergestellt. Der Hash wird daraufhin erneut aktualisiert, um den Einfluss der Figur an ihrem Ursprungsort hinzuzufügen, und falls eine Figur geschlagen wurde, wird deren Hash-Beitrag ebenfalls wieder eingerechnet. Abschließend wird die interne Position der Figur korrigiert, und bei speziellen Zügen wie der Rochade wird durch Aufruf von `resetRookPosition(lastMove)` auch der Turm an die korrekte Position zurückgesetzt. Zum Schluss erfolgt ein Wechsel des aktiven Spielers.

```

1      public void undoLastMove() {
2          if (!moveHistory.isEmpty()) {
3              ChessMove lastMove = moveHistory.pop();
4              updateZobrist(lastMove.getPiece(), lastMove.getToRow(),
5                          lastMove.getToCol());
6              board[lastMove.getFromRow()][lastMove.getFromCol()] =
7                  lastMove.getPiece();

```

```

6         board[lastMove.getToRow()][lastMove.getToCol()] =
          lastMove.getCapturedPiece();
7         updateZobrist(lastMove.getPiece(), lastMove.getFromRow(),
          lastMove.getFromCol());
8         if (lastMove.getCapturedPiece() != null) {
9             updateZobrist(lastMove.getCapturedPiece(),
              lastMove.getToRow(), lastMove.getToCol());
10        }
11        lastMove.getPiece().setRow(lastMove.getFromRow());
12        lastMove.getPiece().setCol(lastMove.getFromCol());
13        if (lastMove.getPiece() instanceof King &&
          Math.abs(lastMove.getToCol() - lastMove.getFromCol()) ==
          2) {
14            resetRookPosition(lastMove);
15        }
16        switchPlayer();
17    }
18 }

```

Listing 14: Methode undoLastMove

5 Evaluation und Test

5.1 Test der Hardwarekomponenten

Nach dem ersten Aufbau der Mensch-Maschine-Schnittstelle traten einige unerwartete Probleme auf. Insbesondere die MUX wiesen Fehlfunktionen auf. Ein erster Fehler bestand darin, dass ein MUX ständig Signale vertauschte, sodass die Eingaben nicht korrekt übermittelt wurden. Zur systematischen Fehlersuche wurde die `printBoard`-Methode eingesetzt, um den aktuellen Zustand der Sensoren visuell darzustellen und fehlerhafte Signalverteilungen zu identifizieren. Nach einigen Tests und Messungen stellte sich heraus, dass ein Kurzschluss zwischen den Leiterbahnen der Adressleitungen der beiden Kanäle des MUX bestand. Dies führte dazu, dass sich die beiden Kanäle gegenseitig beeinflussten und falsche Signale erzeugt wurden. Zur Überprüfung der Leiterbahnen wurden kontinuierlich Widerstände zwischen den Pins gemessen, um Kurzschlüsse oder Unterbrechungen frühzeitig zu erkennen.

Ein Problem war auch, dass ein MUX eine ständige „Signalverwirrung“ aufwies, indem mehrere Kanäle immer einen HIGH Output zurückgaben, selbst wenn nur ein einzelner Sensor aktiviert war. Dieser Fehler konnte dadurch behoben werden, dass festgestellt wurde, dass eine der Adressleitungen „floating“ war. Das bedeutete, dass diese Leitung keine stabile Verbindung aufwies und die Verbindung zur Platine nicht korrekt hergestellt war, was zu einem dauerhaften LOW, also einem 0-Bit in der Schaltabelle des MUX, führte und das Signal verfälschte. Zusätzlich wurden bei der Überprüfung der Messwerte und Spannungen mehrere physisch beschädigte Sensoren entdeckt, deren Widerstandswerte außerhalb der Toleranz lagen. Diese Sensoren wurden daraufhin ausgetauscht, um eine zuverlässige Signalübertragung zu gewährleisten. Nachdem diese Fehler behoben und die Lötstellen überprüft sowie korrekt angesetzt wurden, funktionierte der MUX wieder stabil.

Ein weiteres Problem trat auf, als mehrere Sensoren gleichzeitig ein Output-Signal von sich ga-

ben, obwohl nur ein Sensor einen Input bekommen hatte. Dieser Fehler konnte durch eine genauere Prüfung der Platine gelöst werden, da die MUX und die Sensoren durch eine fehlerhafte Verbindung zwischen den Adressleitungen einen Kurzschluss hatten und nicht korrekt funktionierten. Bei der Analyse zeigten sich zudem sporadisch schwache Magnet-Signale einzelner Felder, die auf mangelnden Kontakt oder unzureichende Magnetstärke zurückzuführen waren. Durch Justieren der Magnetpositionen und Nachlöten kritischer Verbindungen wurde dieses Problem reduziert.

5.2 Test der Softwarekomponenten

Die größte Herausforderung in den Softwaretests lag in der korrekten Handhabung der Spiellogik, insbesondere bei Sonderzügen wie der Rochade. Die für die Validierung und Ausführung der Züge zuständige `processMove`-Methode funktionierte zwar für die meisten regulären Züge, stieß jedoch bei komplexeren Manövern wiederholt auf Probleme. Ein initialer Logikfehler führte dazu, dass das Spielbrett aus der falschen Perspektive interpretiert wurde – bei weißen Zügen reagierte die Software fälschlicherweise aus Sicht von Schwarz. Dies äußerte sich in inkonsistenten Zugvalidierungen und wurde durch eine fehlerhafte Koordinatenzuordnung im Code verursacht. Durch gezieltes Debugging der Board-Repräsentation konnte dieser Fehler lokalisiert und durch Korrektur der Indexberechnungen behoben werden.

Besonders die Rochade bereitete Schwierigkeiten, da ihre Bedingungen nicht immer zuverlässig erkannt wurden, was zu fehlerhaften Spielverläufen führte. Ein zentrales Problem zeigte sich darin, dass bei einem fehlerhaft ausgeführten Zug die Schnittstelle alle weiteren Züge blockierte. Dabei verstand die Software lediglich, dass eine Bewegung stattgefunden hatte, und bewertete den folgenden Zug als ungültig, selbst wenn im realen Spielverlauf eine Korrektur, wie das Zurücksetzen eines Bauern um einen Schritt, vorgenommen wurde. Dieses Verhalten führt dazu, dass manche Fälle, in denen Züge zwar korrigiert, aber von der Software nicht erkannt werden, die Rückverfolgung und Korrektur erheblich erschweren. Zudem traten sporadisch Probleme bei der Erkennung von Zügen auf, wenn Magnet-Signale aufgrund von Positionierungsungenauigkeiten oder Materialschwächen zu schwach ausfielen. Durch eine Anpassung der Signalverarbeitung, einschließlich einer dynamischen Schwellwertanpassung, konnte die Robustheit der Zugererkennung signifikant verbessert werden.

5.3 Bewertung und Zusammenfassung der Testergebnisse

Das entwickelte System zur Digitalisierung von Schachzügen hat seine Kernfunktion – die Erfassung und Darstellung von Spielzügen – grundsätzlich erfolgreich umgesetzt. Die Tests zeigen jedoch sowohl Stärken als auch Grenzen des Prototyps, die im Folgenden zusammengefasst werden.

Die größten Erfolge liegen in der gelungenen Hardware-Software-Integration. Nach der Behebung anfänglicher Probleme wie Kurzschlüssen in den Adressleitungen oder lose gelöteten Sensoren konnte das System Standardzüge zuverlässig erkennen. Bewegungen von Bauern, Läufern oder Springern wurden in den meisten Fällen korrekt digitalisiert und in der grafischen Benutzeroberfläche angezeigt. Die selbst programmierte Software übersetzte die erfassten Daten erfolgreich in eine digitale Partienotation, was besonders für Trainingszwecke oder Partieanalysen nützlich ist. Die Fehlerbehebung mithilfe des Multimeters und der `printBoard`-Methode erwies sich als effektiv, um Hardwareprobleme systematisch zu isolieren und zu beheben.

Allerdings traten Herausforderungen bei komplexeren Anforderungen auf. Sonderzüge wie die Rochade oder der „en passant“-Schlag bereiteten der Software anfangs Schwierigkeiten, da die Implementierung der Schachregeln nicht alle Randfälle abdeckte. Erst durch mehrfache Überarbeitung des Codes – etwa die Korrektur der Koordinatenindizes und die Verbesserung der Validierungslogik – konnten diese Züge zuverlässiger erkannt werden. Ein weiterer kritischer Punkt war die mechanische Empfindlichkeit der Hardware: Leicht verschobene Sensoren führten zu Fehldetektionen. Dies erfordert in der Praxis sorgfältiges Positionieren der Figuren, was das Spielerlebnis beeinträchtigen kann.

Die Stabilität des Systems erwies sich als zweischneidig. Zwar arbeitete die Hardware nach Abschluss aller Reparaturen stabil, doch die manuellen Nachjustierungen (z. B. Nachlöten von Kontakten) zeigten, dass das Design für den Dauereinsatz noch zu anfällig ist. Auch die Software zeigte Schwächen bei der Fehlerrückverfolgung: Wurde ein Zug physisch korrigiert (z. B. durch Zurücksetzen einer Figur), erkannte das System dies nicht immer automatisch, was zu Schwierigkeiten im Spielfluss führt.

Zusammenfassend lässt sich sagen, dass der Prototyp alle grundlegenden Anforderungen eines digitalen Schachbretts erfüllt, aber noch nicht ausgereift ist. Die Stärken liegen in der kostengünstigen Umsetzung und der gelungenen Verbindung von analogem Spielgefühl mit digitaler Darstellung. Die Schwächen – vor allem die eingeschränkte Regelabdeckung und mechanische Sensibilität – verdeutlichen, warum professionelle Lösungen komplexer und teurer sind. Für ein Schulprojekt ist das Ergebnis dennoch ein trotz Problemen ein Erfolg.

6 Zusammenfassung und Ausblick

Im Rahmen dieses Projekts wurde ein Schachbrett entwickelt, das in der Lage ist, Spielzüge automatisch zu erfassen und digital zu verarbeiten. Zu Beginn stand die Idee im Fokus, eine Mensch-Maschine-Schnittstelle zu schaffen, die analoge Schachzüge mit minimalem externem Eingriff erkennt. Dafür wurden unter den einzelnen Feldern Magnetsensoren platziert, um zuverlässig zu detektieren, ob ein Feld besetzt ist oder nicht.

die Schnittstelle war in der Lage, viele Standardzüge korrekt zu erkennen und den Spielverlauf digital zu speichern. Insbesondere für einfache Zugfolgen funktionierte das Prinzip zuverlässig. Dennoch traten mit zunehmender Spieldauer Probleme auf: Sonderzüge wie Rochade oder Bauernumwandlungen führten zu Fehlinterpretationen, und die Fehleranfälligkeit der Schnittstelle nahm zu. Besonders problematisch war die fehlende Möglichkeit, Schachfiguren eindeutig zu unterscheiden. Dies erschwerte die Korrektur fehlerhafter Züge erheblich, da die Schnittstelle nicht nachvollziehen konnte, welche Figur ursprünglich an einer bestimmten Position stand. Fehlerhafte Erkennungen summierten sich dadurch im Verlauf der Partie, was letztlich zu Diskrepanzen zwischen der digitalen Spiellogik und der realen Brettstellung führte.

Trotz dieser Herausforderungen war das Projekt eine äußerst lehrreiche Erfahrung. Die Entwicklung einer eigenen Hardware-Lösung und deren Integration in eine Software war ein faszinierender Prozess, bei dem zahlreiche technische und konzeptionelle Herausforderungen bewältigt werden mussten. Ich konnte wertvolle Erkenntnisse über die Signalverarbeitung von Sensoren, Störsignale in elektronischen Schaltungen sowie die komplexen Wechselwirkungen zwischen Hardware und

Software gewinnen. Besonders spannend war es zu beobachten, wie eine physische Aktion – das Bewegen einer Schachfigur – in einem digitalen System abgebildet wurde. Auch wenn nicht alle Aspekte der Schnittstelle reibungslos funktionierten, war es ein beeindruckender Moment, wenn ein korrekt erkannter Zug auf dem Bildschirm dargestellt wurde.

Dieses Projekt war ein wichtiger Prototyp, der als Grundlage für zukünftige Iterationen dienen kann. Ein zentrales Ziel zukünftiger Versionen wäre die Implementierung der Figurenidentifikation, um eine zuverlässigere Fehlerkorrektur zu ermöglichen. Dafür könnten verschiedene Ansätze in Betracht gezogen werden.

Ein weiterer interessanter Ansatz zur Verbesserung der Spielvalidierung wäre der Einbau von Elektromagneten in die Felder des Schachbretts. Diese könnten Figuren bei Bedarf magnetisch fixieren, um illegale Züge sofort zu korrigieren. Wenn ein Spieler versucht, eine Figur entgegen den Schachregeln zu bewegen, würde das System die Magnetkraft aktivieren und die Figur am ursprünglichen Feld halten. Gleichzeitig könnte ein solches System verhindern, dass Spieler außerhalb ihrer Zugzeit Figuren bewegen – die Magnete wären nur für den aktiven Spieler deaktiviert. Dies würde nicht nur regelwidrige Züge blockieren, sondern auch das Spieltempo verbessern, indem versehentliches Verschieben von Figuren während der Denkpause unterbunden wird.

Für die Fehlerkorrektur ließe sich dieses Konzept erweitern: Bei Erkennung eines ungültigen Zuges durch die Software könnten die Magnete automatisch reaktiviert werden, um die Figur in ihre Ausgangsposition zurückzuzwingen. Kombiniert mit einer visuellen Warnung auf dem digitalen Interface entstünde so ein robustes Fehlerfeedback-System. Die Hardware-Implementierung würde allerdings neue Herausforderungen mit sich bringen, etwa die präzise Ansteuerung von 64 Elektromagneten und die Integration in die bestehende Sensorik.

Ein weiterer sinnvoller Zusatz wäre die Möglichkeit, gespeicherte Spielstände später fortzusetzen. Durch die Archivierung von Brettstellungen in einer Datenbank könnten Nutzer gezielt Schlüsselmomente einer Partie speichern – etwa komplexe Mittelspielpositionen oder Endspielszenarien –, um diese zu Trainingszwecken erneut aufzurufen. Ambitionierte Spieler hätten so die Chance, Alternativzüge zu testen oder Fehler retrospektiv zu analysieren. Praktisch ließe sich dies sogar auf unterbrochene Partien erweitern. Während diese Funktionalität den Rahmen der aktuellen Implementierung überschreitet, würde sie das Schachbrett zu einem flexiblen Lernwerkzeug machen.

Zusammenfassend hat mir dieses Projekt nicht nur wertvolle technische Erkenntnisse vermittelt, sondern auch gezeigt, dass Fehler und unerwartete Herausforderungen essenzielle Bestandteile des Entwicklungsprozesses sind. Die kontinuierliche Verbesserung eines Systems durch iterative Optimierung ist ein spannender und lohnender Prozess. Trotz der bestehenden Herausforderungen hat mir die Arbeit an dieser Mensch-Maschine-Schnittstelle große Freude bereitet, und ich sehe in der Weiterentwicklung dieser Idee viel Potenzial für zukünftige Projekte.

7 Quellenverzeichnis

Abbildungsverzeichnis

1	Grundstellung des Schachspiels. (eigens erstellte Abbildung)	6
2	Der weiße Bauer zieht von d2 nach d3 und zieht beim erste Zug des Bauern von d2 nach d4. (eigens erstellte Abbildung)	7
3	Der weiße Springer zieht von b1 nach c3. (eigens erstellte Abbildung)	7
4	Der weiße Läufer zieht von f1 nach d3. (eigens erstellte Abbildung)	8
5	Der schwarze Turm zieht von h8 nach h5. (eigens erstellte Abbildung)	8
6	Die weiße Dame, zieht von d1 nach f3 und weiße Dame, zieht von f3 nach f5. (eigens erstellte Abbildung)	9
7	Lange und kurze Rochade (eigens erstellte Abbildung)	9
8	der weiße Bauer schlägt Schwarz auf Diagonal auf d5. (eigens erstellte Abbildung) .	10
9	Beispielstellung für En Passant: Der weiße Bauer schlägt den schwarzen Bauern auf c5 im Vorübergehen. (eigens erstellte Abbildung)	10
10	Übersicht der Arduino Mega-Platine mit den wichtigsten Anschlüssen Arduino_Mega .	11
11	Schematische Darstellung eines digitalen unipolaren Hallsensors. Quelle: Hallsensoren_Arduino_projekt	12
12	Adresstabelle des 16:1-Multiplexers. Die vier Adressbits (S_3 bis S_0) bestimmen den aktiven Kanal C_0 bis C_{15} , Mux_tabelle	12
13	Schaltplan des 16:1-Multiplexers CD74HC406, Analog_Digital_MUX	13
14	Ein herkömmlichen Holzschachbrett, das als Basis für die Schnittstellenentwicklung gewählt wurde.	13
15	3D-Modelle der Schachfiguren mit angepasster Befestigung für Magneten 3D_Figuren	14
16	UML-Diagramm der Softwarearchitektur	16
17	Vollendete Hallsensoren-Matrix	18
18	Vollendeter Prototyp der Schnittstelle	18